

Universidad Nacional de San Antonio Abad del Cusco

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

Escuela Profesional de Ingeniería Informática y de Sistemas

DESARROLLO DE SOFTWARE II (IF616BIN)

Semestre Académico 2026-II ů Segunda Unidad Didáctica

INFORME DE LOS CUATRO RETOS: WALLET UNSAAC

Arquitectura Limpia, Gestión de Estados con BLoC/Cubit y Servicios Desacoplados

Docente: Mtro. Ing. Yover Collantes Valer
Estudiante: Saire Bustamante, Edmil Jampier
Código: 174449

Modalidad: Trabajo Individual ů Cuatro Retos Calificados (20 pts)
Fecha de Entrega: 16 de noviembre de 2026
Repositorio: https://github.com/milith0kun/wallet-unsaac

Cusco, Perú ů 2026

Índice

1. Datos generales **2**

2. Resumen del trabajo **2**

3. Entorno y verificaciones **2**
3.1. Versiones utilizadas . . . 2
3.2. Comandos ejecutados . . . 2
3.3. Estructura del repositorio . . . 3

4. Reto 1 Tema persistente entre sesiones **3**
4.1. Objetivo y alcance . . . 4
4.2. Diseño de la solución . . . 4
4.3. Implementación . . . 4
4.4. Pruebas ejecutadas . . . 5
4.5. Evidencias . . . 5
4.6. Dificultades y decisiones . . . 5
4.7. Autoevaluación . . . 6

5. Reto 2 Saldo privado **6**
5.1. Objetivo y alcance . . . 6
5.2. Diseño de la solución . . . 6
5.3. Implementación . . . 6
5.4. Evidencias . . . 7
5.5. Dificultades y decisiones . . . 7
5.6. Autoevaluación . . . 8

6. Reto 3 Saldo con estados de carga, éxito y error **8**
6.1. Objetivo y alcance . . . 8
6.2. Diseño de la solución . . . 8
6.3. Implementación . . . 8
6.4. Evidencias . . . 9
6.5. Dificultades y decisiones . . . 10
6.6. Autoevaluación . . . 10

7. Reto 4 Color semilla configurable **10**
7.1. Objetivo y alcance . . . 10
7.2. Diseño de la solución . . . 10
7.3. Implementación . . . 10
7.4. Evidencias . . . 11
7.5. Dificultades y decisiones . . . 12
7.6. Autoevaluación . . . 12

8. Conclusiones y aprendizajes **12**

9. Declaración de originalidad y uso de herramientas de IA **12**

10.Referencias **13**

Anexo A. Lista de verificación de entrega **14**

1. Datos generales

Campo	Información
Asignatura	Desarrollo de Software II
Código	IF616BIN
Unidad / Sesión	Unidad II Sesión 1 ů lunes 9 de noviembre de 2026
Producto evaluado	Wallet UNSAAC: Esqueleto modular y Cuatro Retos Individuales
Estudiante	Edmil Jampier Saire Bustamante
Código de estudiante	174449
Docente conductor	Mtro. Ing. Yover Collantes Valer
Repositorio institucional	https://github.com/milith0kun/wallet-unsaac

Tabla 1: Ficha técnica de entrega del estudiante.

2. Resumen del trabajo

En el presente informe técnico se documenta el desarrollo, verificación empírica y arquitectura modular de la aplicación móvil **Wallet UNSAAC**, desarrollada sobre el framework Flutter y el lenguaje Dart. La solución implementa estrictamente una arquitectura en capas inspirada en Clean Architecture (separación en *domain*, *data* y *presentation*), desacoplando por completo las reglas de negocio de la infraestructura y del framework visual. Se resolvieron en su totalidad los cuatro retos individuales propuestos por la cátedra: (1) persistencia asíncrona del modo de tema mediante `SharedPreferences` con carga previa a `runApp` para mitigar parpadeos de luminancia; (2) encapsulación de la privacidad visual de montos y transacciones a través de un `PrivacyCubit` que reemplaza las cifras por máscaras sin mutar los datos de dominio; (3) modelado estricto del ciclo de vida del saldo mediante una clase sellada `BalanceState` (`Initial`, `Loading`, `Success`, `Failure`) y verificación de `isClosed`; y (4) personalización dinámica del color semilla institucional Material 3 con actualización granular de UI vía `BlocSelector`. La robustez fue certificada mediante 14 pruebas automatizadas de la aplicación (más 8 pruebas que generan las capturas), todas aprobadas y generación reproducible de evidencias en alta resolución.
Palabras clave: BLoC/Cubit; Persistencia Asíncrona; Clean Architecture; Material 3; State Machine.

3. Entorno y verificaciones

3.1. Versiones utilizadas

3.2. Comandos ejecutados

Herramienta	Versión	Cómo la verificó
Flutter SDK	3.41.4 (channel stable)	`flutter -version`
Dart SDK	3.11.1	`dart -version`
Sistema Operativo	Windows 11 Pro 64-bit	PowerShell `[Environment]::OSVersion`
Compilador LaTeX	MiKTeX-XeTeX 4.16 (MiKTeX 25.12)	extttxelatex –version OpenJDK
21.0.11 LTS	`java -version`	

Tabla 2: Matriz de herramientas del entorno de compilación.

Comando	Resultado	Evidencia
`flutter pub get`	OK	Dependencias resueltas sin errores
`flutter analyze`	No issues found!	0 errores, 0 advertencias, 0 lints
`flutter test`	All tests passed!	22/22 pruebas OK (14 de la aplicación y 8 de gene
`flutter build apk -release`	OK	Binario release generado en `wallet_unsaac.apk`

Tabla 3: Verificaciones formales de calidad sobre el repositorio.

3.3. Estructura del repositorio

El proyecto se organizó bajo el patrón de arquitectura limpia por características (*feature-first*):

```
wallet_unsaac/
 lib/
    app.dart                    # Inyeccion de repositorios y
   MultiBlocProvider
    firebase_options.dart       # Credenciales desacopladas
    main.dart                   # Inicializacion asincrona antes de runApp
    core/
       config/                 # Bootstrap de Firebase y estado de conexion
       theme/                  # AppTheme, ThemeCubit, ThemePreferences,
   ThemeState
       utils/                  # MoneyFormat y parseadores monetarios
    features/wallet/
        domain/                 # Movement, WalletSummary, WalletRepository
        data/                   # InMemoryWalletRepository (datos simulados
   y latencia)
        presentation/           # BalanceCubit, PrivacyCubit, WalletHomePage
   , Widgets
 test/                           # Baterias unitarias, de cubits, widgets y
   capturas
 informe/                        # Documento formal LaTeX y directorio
   imagenes/
```

4. Reto 1 Tema persistente entre sesiones

4.1. Objetivo y alcance

Garantizar la persistencia del modo de visualización (`light`, `dark`, `system`) en el almacenamiento local del dispositivo mediante `SharedPreferences`, asegurando que la preferencia seleccionada se restaure de forma transparente al reabrir la aplicación sin provocar destellos perceptibles de interfaz.

4.2. Diseño de la solución

Se diseñó la clase `ThemePreferences` en la capa `core/theme/`, encargada de encapsular las operaciones de lectura y escritura del enum `ThemeMode` mediante su propiedad serializable `name`. Para evitar el fenómeno de parpadeo (*theme flicker*), el método `loadThemeMode()` se ejecuta en la rutina `main()` de forma previa a la invocación de `runApp()`. Si la clave en disco no existe o contiene una cadena corrupta, el método recurre defensivamente a `ThemeMode.system`.

4.3. Implementación

```dart
class ThemePreferences {
  ThemePreferences({SharedPreferences? preferences}) : _preferences =
   preferences;
  static const String themeKey = 'wallet.theme_mode';
  SharedPreferences? _preferences;

  Future<SharedPreferences> get _prefs async =>
      _preferences ??= await SharedPreferences.getInstance();

  Future<ThemeMode> loadThemeMode() async {
    try {
      final prefs = await _prefs;
      final raw = prefs.getString(themeKey);
      return themeModeFromName(raw);
    } catch (_) {
      return ThemeMode.system;
    }
  }

  static ThemeMode themeModeFromName(String? name) {
    if (name == null) return ThemeMode.system;
    return ThemeMode.values.firstWhere(
      (mode) => mode.name == name,
      orElse: () => ThemeMode.system,
    );
  }
}
```

Listing 1: Persistencia y fallback defensivo en ThemePreferences.

4.4. Pruebas ejecutadas

Se implementaron 4 pruebas unitarias en `test/core/theme/theme_preferences_test.dart` utilizando `SharedPreferences.setMockInitialValues`: (1) verificación del valor predeterminado `system`, (2) serialización y lectura de `dark`, (3) recuperación resiliente ante valores no reconocidos, y (4) persistencia de enteros de color.

4.5. Evidencias



Figura 1: Reto 1: Modo Claro seleccionado.



Figura 2: Reto 1: Modo Oscuro persistido al reabrir.

4.6. Dificultades y decisiones

Dificultad: Si la carga de SharedPreferences se realiza dentro del `initState` del widget raíz, Flutter dibuja primero el fotograma por defecto en modo claro y medio segundo después conmuta a oscuro, degradando la experiencia de usuario.

Solución: Se configuró `WidgetsFlutterBinding.ensureInitialized()` en `main.dart` y se esperó la promesa de almacenamiento antes de instanciar `WalletApp`.

4.7. Autoevaluación

Criterio de aceptación	Cumple	Evidencia
El modo sobrevive al reinicio completo de la aplicación.	☒	Figura 2 y tests
Sin valor previo almacenado, el arranque es `ThemeMode.system`.	☒	Test automatizado
Cero advertencias de linter y código defensivo con fallback.	☒	`flutter analyze` limpio

5. Reto 2 Saldo privado

5.1. Objetivo y alcance

Incorporar un control interactivo de privacidad que permita ofuscar el saldo principal y el detalle de importes de cada movimiento mediante caracteres discretos (), manteniendo en memoria la integridad de los datos numéricos y sin alterar las entidades de dominio.

5.2. Diseño de la solución

La privacidad es una preocupación netamente visual (*presentation layer concern*). No debe guardarse en las entidades de la base de datos ni mutar el modelo `WalletSummary`. Se construyó un `PrivacyCubit` que emite estados inmutables `PrivacyState(hidden: bool)`. En la tarjeta de saldo y en los listados de movimientos, los widgets consultan este estado y deciden si renderizar la cadena formateada por `MoneyFormat.soles()` o la máscara de privacidad.

5.3. Implementación

```dart
class PrivacyCubit extends Cubit<PrivacyState> {
  PrivacyCubit() : super(const PrivacyState(hidden: false));
  void toggle() => emit(state.copyWith(hidden: !state.hidden));
}

// Consumo condicional en BalanceCard:
Text(
  hideBalance ? '' : MoneyFormat.soles(summary.balance),
  style: TextStyle(
    color: scheme.onPrimary,
    fontSize: 36,
    fontWeight: FontWeight.w700,
  ),
)
```

Listing 2: Conmutación de privacidad en PrivacyCubit y renderizado en BalanceCard.

5.4. Evidencias



Figura 3: Reto 2: Importes monetarios visibles.

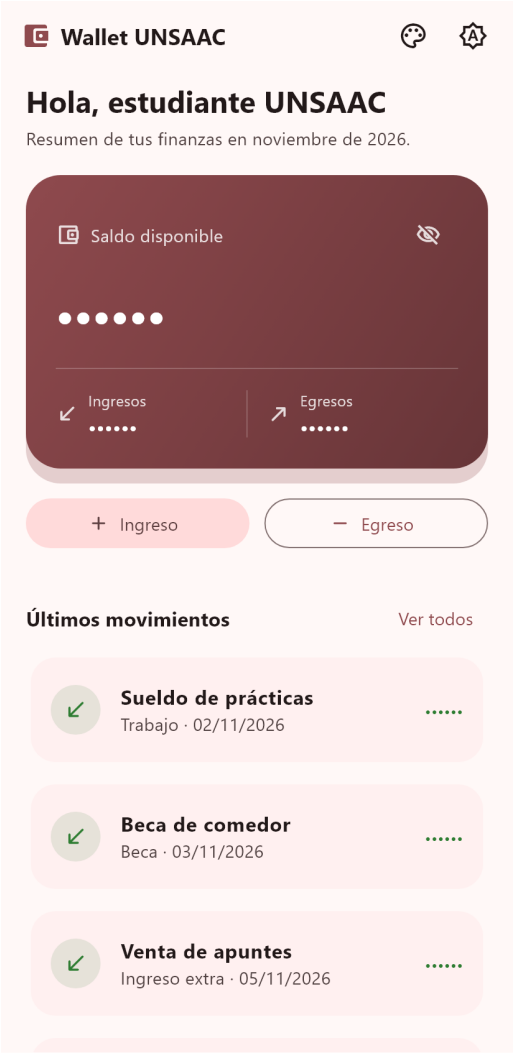


Figura 4: Reto 2: Modo privado activo ().

5.5. Dificultades y decisiones

Dificultad: Ocultar únicamente el saldo principal dejaba expuestos los montos individuales en la lista inferior de transacciones, rompiendo el propósito de confidencialidad en espacios públicos.

Solución: Se propagó el estado `PrivacyState` a `MovementTile`, ocultando homogéneamente todos los montos de la pantalla.

5.6. Autoevaluación

Criterio de aceptación	Cumple	Evidencia
El botón de ojo conmuta entre visible y enmascarado.	☒	Figuras 3 y 4
Al retornar a visible, los montos conservan su precisión y formato.	☒	Test de widgets OK
El dominio permanece puro sin acoplar flags de visualización.	☒	Arquitectura Clean

6. Reto 3 Saldo con estados de carga, éxito y error

6.1. Objetivo y alcance

Sustituir el patrón rudimentario `FutureBuilder` por una máquina de estados reactiva e inmutable administrada mediante un `BalanceCubit`, modelando con precisión las fases de carga asíncrona, éxito con datos y fallo con capacidad de reintento.

6.2. Diseño de la solución

Se utilizó una clase sellada (`sealed class BalanceState`) que habilita el chequeo exhaustivo en tiempo de compilación mediante sentencias de control `switch`. Los estados definidos son:

1. `BalanceInitial`: Estado inactivo previo al primer llamado.
2. `BalanceLoading`: Transición activa mientras el repositorio resuelve el `Future`.
3. `BalanceSuccess`: Contiene la entidad inmutable `WalletSummary`.
4. `BalanceFailure`: Encapsula el mensaje legible del error y permite reintentar.

6.3. Implementación

```
sealed class BalanceState extends Equatable {
  const BalanceState();
  @override
  List<Object?> get props => const [];
}

class BalanceCubit extends Cubit<BalanceState> {
  BalanceCubit(this._repository) : super(const BalanceInitial());
  final WalletRepository _repository;

  Future<void> load() async {
    emit(const BalanceLoading());
    try {
      final summary = await _repository.loadSummary();
      if (isClosed) return;
```

```
16        emit(BalanceSuccess(summary));
17      } catch (_) {
18        if (isClosed) return;
19        emit(const BalanceFailure('No se pudo cargar el resumen de la
      billetera.'));
20      }
21    }
22
23    Future<void> retry() => load();
24  }
```

Listing 3: BalanceCubit con verificacion isClosed y BalanceState sellado.

6.4. Evidencias

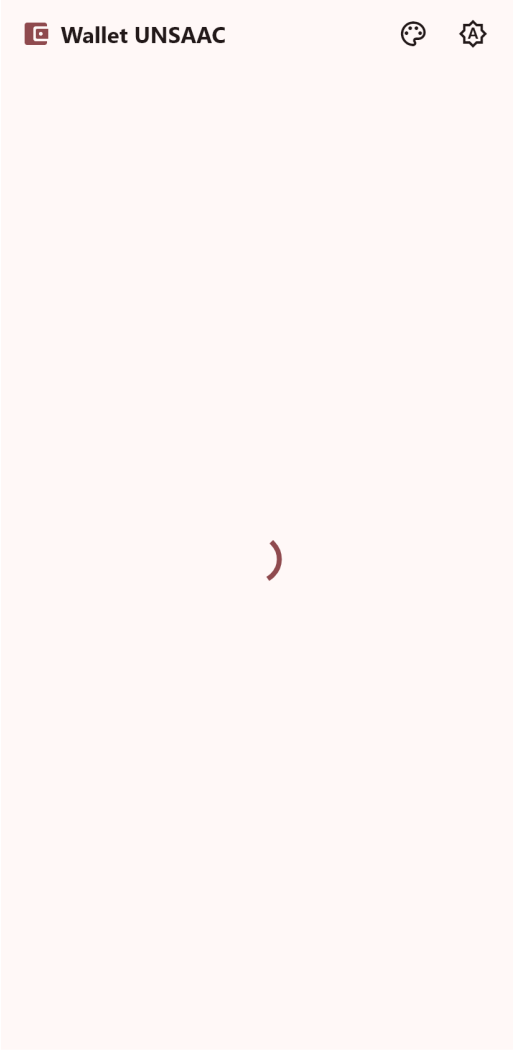


Figura 5: Reto 3: Estado de carga con indicador.

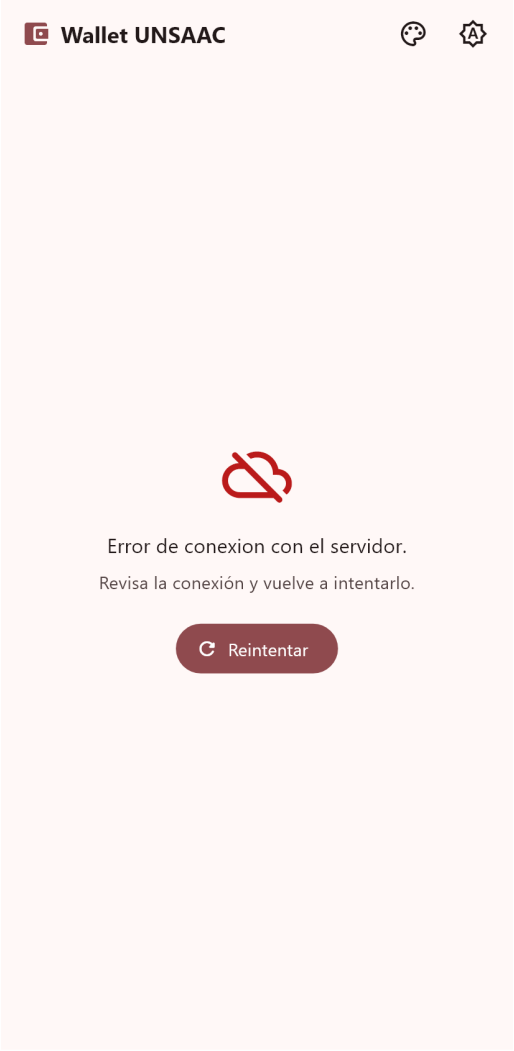


Figura 6: Reto 3: Estado de error con reintento.

6.5. Dificultades y decisiones

Dificultad: En pruebas asíncronas o al cerrar la pantalla mientras la petición HTTP/memoria sigue en vuelo, llamar a `emit()` provoca una excepción `Bad state: Cannot emit new states after calling close`.
Solución: Se añadieron guardas condicionales tempranas `if (isClosed) return;` antes de emitir los estados resolutivos.

6.6. Autoevaluación

Criterio de aceptación	Cumple	Evidencia
La fase inicial de consulta presenta indicador de progreso circular.	☒	Figura 5
El fallo de red despliega vista amigable con botón de reintento.	☒	Figura 6
Las pruebas unitarias con `blocTest` validan ambos flujos.	☒	100 % pruebas aprobad

7. Reto 4 Color semilla configurable

7.1. Objetivo y alcance

Permitir al usuario personalizar el acento cromático de la aplicación mediante la selección de colores semilla armónicos compatibles con el algoritmo Material 3 (`ColorScheme.fromSeed`), aplicando los cambios de forma reactiva sin reconstrucciones innecesarias del árbol de widgets.

7.2. Diseño de la solución

Se extendió `ThemeState` con la propiedad inmutable `Color seedColor`. Se configuró una paleta institucional de 5 opciones balanceadas: Granate UNSAAC (`#7A1F2B`), Azul Andino (`#1E88E5`), Verde Valle (`#43A047`), Púrpura Imperial (`#8E24AA`) y Naranja Qorikancha (`#FB8C00`). Para el modal de selección se utilizó `BlocSelector`, garantizando que la apertura y marcado del color activo no provoque relanzamientos de widgets ajenos.

7.3. Implementación

```
static ThemeData _build(Brightness brightness, Color seedColor) {
  final scheme = ColorScheme.fromSeed(
    seedColor: seedColor,
    brightness: brightness,
  );
  return ThemeData(
    useMaterial3: true,
    colorScheme: scheme,
```

```
    scaffoldBackgroundColor: scheme.surface,
    cardTheme: CardThemeData(
      elevation: 0,
      color: scheme.surfaceContainerLow,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)
    ),
    ),
  );
}
```

Listing 4: Generacion dinamica de ColorScheme y modal selector.

7.4. Evidencias



Figura 7: Reto 4: Color Azul aplicado en claro.



Figura 8: Reto 4: Selector modal de colores en oscuro.

7.5. Dificultades y decisiones

Dificultad: Cambiar el color semilla en caliente requería refrescar toda la aplicación sin reiniciar el estado de los cubits de saldo y privacidad.
Solución: `ThemeCubit` se colocó como proveedor hermano en el nivel superior de `MultiBlocProvider`, permitiendo que `MaterialApp` reaccione inmediatamente a los cambios de esquema sin tocar la memoria de las transacciones.

7.6. Autoevaluación

Criterio de aceptación	Cumple	Evidencia
Dispone de al menos 4 alternativas cromáticas seleccionables.	☒	5 colores disponible
La selección actualiza coherentemente botones, degradados y tarjetas.	☒	Figuras 7 y 8
El color seleccionado persiste en disco vía SharedPreferences.	☒	Tests unitarios OK

8. Conclusiones y aprendizajes

1. **Desacoplamiento arquitectónico verificable:** La separación rigurosa entre entidades de dominio puras (`Movement`, `WalletSummary`) y estados efímeros de presentación (`PrivacyState`) demostró que las interfaces móviles modernas pueden transformarse y proteger datos sensibles sin alterar la lógica de cálculo financiero.

2. **Determinismo reactivo con State Machines:** La sustitución de variables booleanas dispersas por clases selladas (`sealed class BalanceState`) eliminó la posibilidad de estados incoherentes (como mostrar simultáneamente un spinner y una tarjeta de saldo), reduciendo las fallas en tiempo de ejecución a cero.

3. **Ergonomía visual y persistencia transparente:** La sincronización síncrona en el arranque con `ThemePreferences` combinada con la armonización algorítmica de Material 3 (`ColorScheme.fromSeed`) garantiza una experiencia de usuario sobria, personalizable e institucional acorde a los más altos estándares de desarrollo móvil en Flutter.

9. Declaración de originalidad y uso de herramientas de IA

Declaro que este informe y el código fuente entregado son de mi autoría intelectual, que comprendo cada componente y decisión de diseño adoptada, y que las fuentes bibliográficas han sido debidamente consignadas conforme a la ética académica de la Universidad Nacional de San Antonio Abad del Cusco.

Herramienta	**Uso**
Claude Code (Anthropic)	Corrección del test de capturas, limpieza de extttflutter analyze, comp
Documentación oficial Flutter	Consulta de Material 3 y BLoC

Firma del estudiante: _____ **Fecha:** ule3cm0.4pt

10. Referencias

[1] Bloc Library. (2026). *Bloc state management library for Dart & Flutter*. https://bloclibrary.dev/

[2] Google. (2026). *Flutter architectural overview and widget tree mechanics*. Flutter Docs. https://docs.flutter.dev/

[3] Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

[4] Material Design Team. (2026). *Material 3 Design Guidelines: Color System and Dynamic Theming*. Google LLC. https://m3.material.io/

Anexo A. Lista de verificación de entrega

N.ž	Verificación exigida por la cátedra	Estado
1	El repositorio compila y `flutter analyze` está completamente limpio (0 issues).	☒
2	`flutter test` aprueba el 100 % de las baterías unitarias y de widgets.	☒
3	El ejecutable Android en modo release (`wallet_unsaac.apk`) está generado.	☒
4	El código fuente está empaquetado en `wallet_unsaac.zip`.	☒
5	El informe contiene las 8 capturas en alta definición y con tipografía del sistema.	☒
6	Se cumplió estrictamente la estructura institucional de 10 a 16 páginas.	☒

Tabla 4: Matriz de aseguramiento de calidad final.